Last part of Unit 4

- In order to monitor sequences and other behavioural aspects of a design for *functional coverage*, cover property statements can be used.
- The syntax of these is the same as that of assert property.
- The simulator keeps a count of the number of times the property in the cover property statement holds or fails.
- This can be used to determine whether or not certain aspects of the designs functionality have been exercised.

```
module Amod2(input bit clk);
        bit X, Y;
        sequence s1; @(posedge clk)
                X ##1 Y;
        endsequence
CovLavel: cover property (s1);
... endmodule
```

What is the difference between cover and assert property?

- The difference is that **cover**ing a property ignores the **failures**, and **assert**ing a property ignores the **passes**.
- Actually, you have a choice with an assert, you may want to know that it passed *at least once*, and never fails.
- An assertion that fails usually invalidates the entire test and all associated coverage you have collected for that test.

Binding

- When RTL is already written and it becomes responsibility of a verification engineer to add assertion. And RTL designer does not want verification engineer to modify his RTL for the sake of adding assertion then, **bind** feature of SystemVerilog comes for rescue.

- One can write all the assertion he or she wants to write in a seperate file and using bind, he or she can bind the ports of his assertion file with the port/signals of the RTL in his testbench code.

- A Bind feature can be used in following places.
  - module
  - interface
  - compilation unit scope

- There are two forms of binding
  - Multi Instance : In this form, binding is done to multiple instance of a module.
  - Single Instance : In this form, binding is done to single instance of a module.

Binding

- We have seen that assertions can be included directly in the source code of the modules in which they apply.
- They can even be embedded in procedural code. Alternatively, verification code can be written in a separate program, for example, and that program can then be bound to a specific module or module instance.

- For example,
  - suppose there is a module for which assertions are to be written:
    - module M (...); // The design is modelled here
    - endmodule
  - The properties, sequences and assertions for the module can be written in a separate program:
    - program M_assertions(...); // sequences, properties, assertions for M go here
    - endprogram
  - This program can be bound to the module M like this:
    - bind M M_assertions M_assertions_inst (...);
  - The syntax and meaning of M_assertions is the same as if the program were instanced in the module itself:
    - module M (...);
    - // The design is modelled here
    - M_assertions M_assertions_inst (...);
    - endmodule

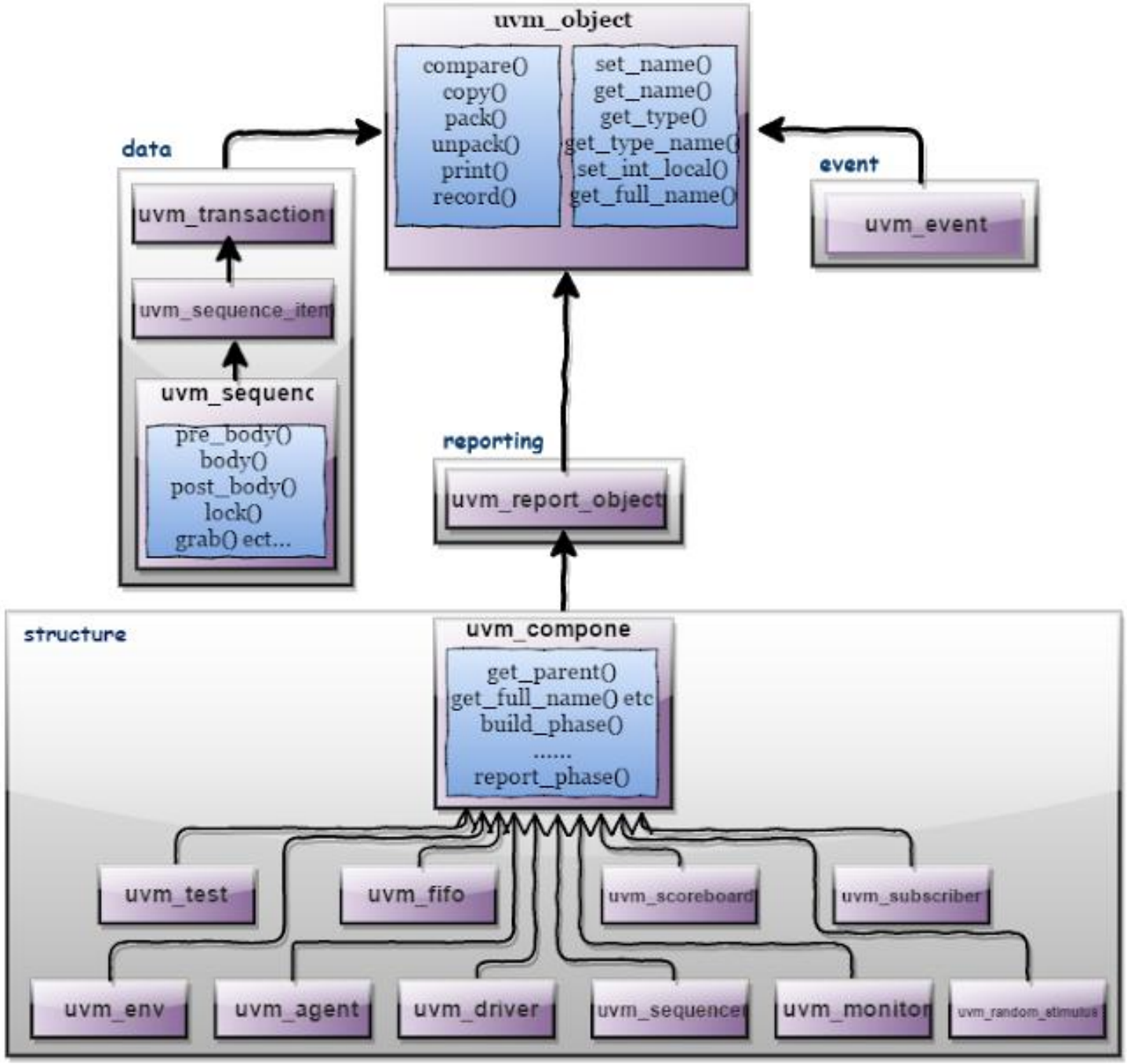
Unit 5

- **Building Test bench:** Layered test bench architecture, Introduction to Universal verification methodology, Overview of UVM, Base classes and simulation phases in UVM and UVM macros, Unified messaging in UVM, UVM environment structure, Connecting DUT-Virtual Interface.

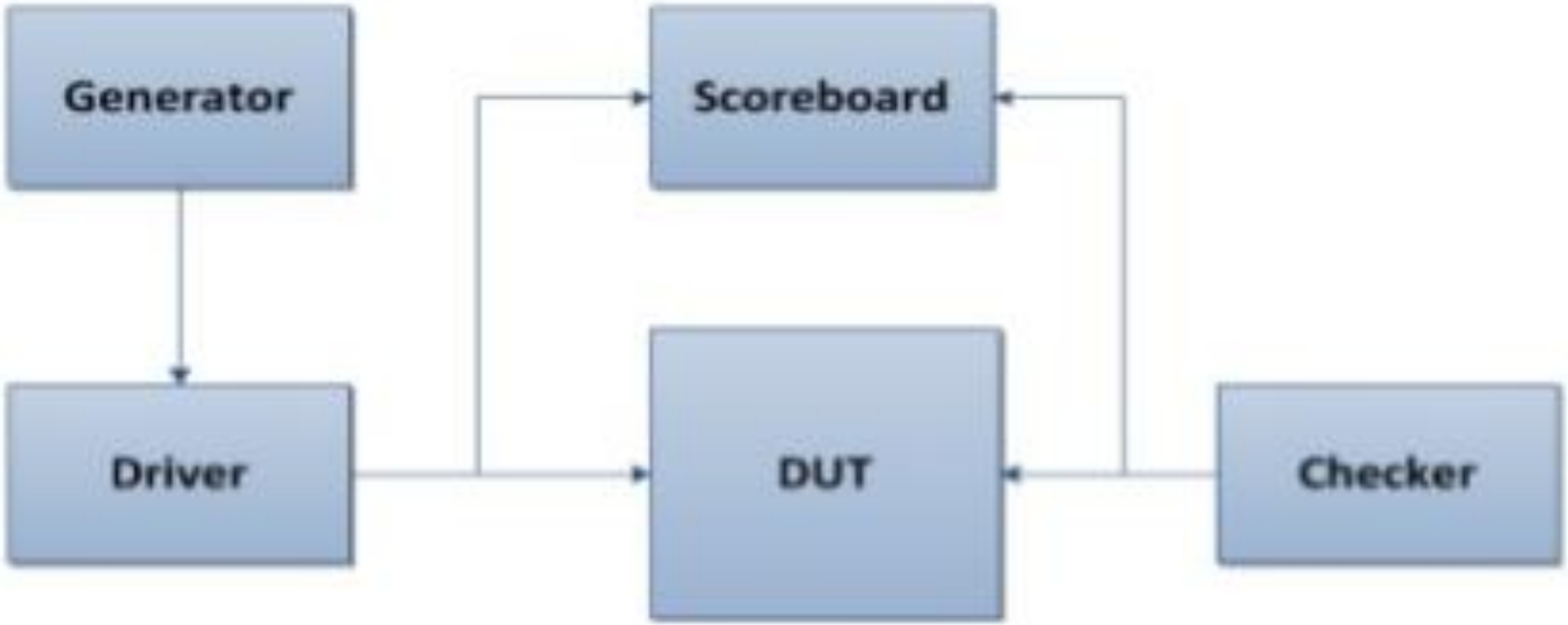
- The **U**niversal **V**erification **M**ethodology (**UVM**) consists of class libraries needed for the development of well constructed, reusable SystemVerilog based Verification environment.
- In simple words, UVM consists of set of base classes with methods defined in it, SystemVerilog verification environment can be developed by extending these base classes.

- UVM consists of three main types of UVM classes,
  - uvm_object
    - Core class based operational methods (create, copy, clone, compare, print, record, etc..), instance identification fields (name, type name, unique id, etc.) and random seeding were defined in it.
    - All uvm_transaction and uvm_component were derived from uvm_object.
  - uvm_transaction
    - Used in stimulus generation and analysis.
  - uvm_component
    - Components are quasi-static objects that exist throughout simulation.
    - Every uvm_component is uniquely addressable via a hierarchical path name, e.g. “env.agent.driver”.
    - The uvm_component also defines a phased test flow, that components follow during the course of simulation. Each phase(build, connect, run, etc.) is defined by a callback that is executed in precise order.
    - The uvm_component also defines configuration, reporting, transaction recording, and factory interfaces.

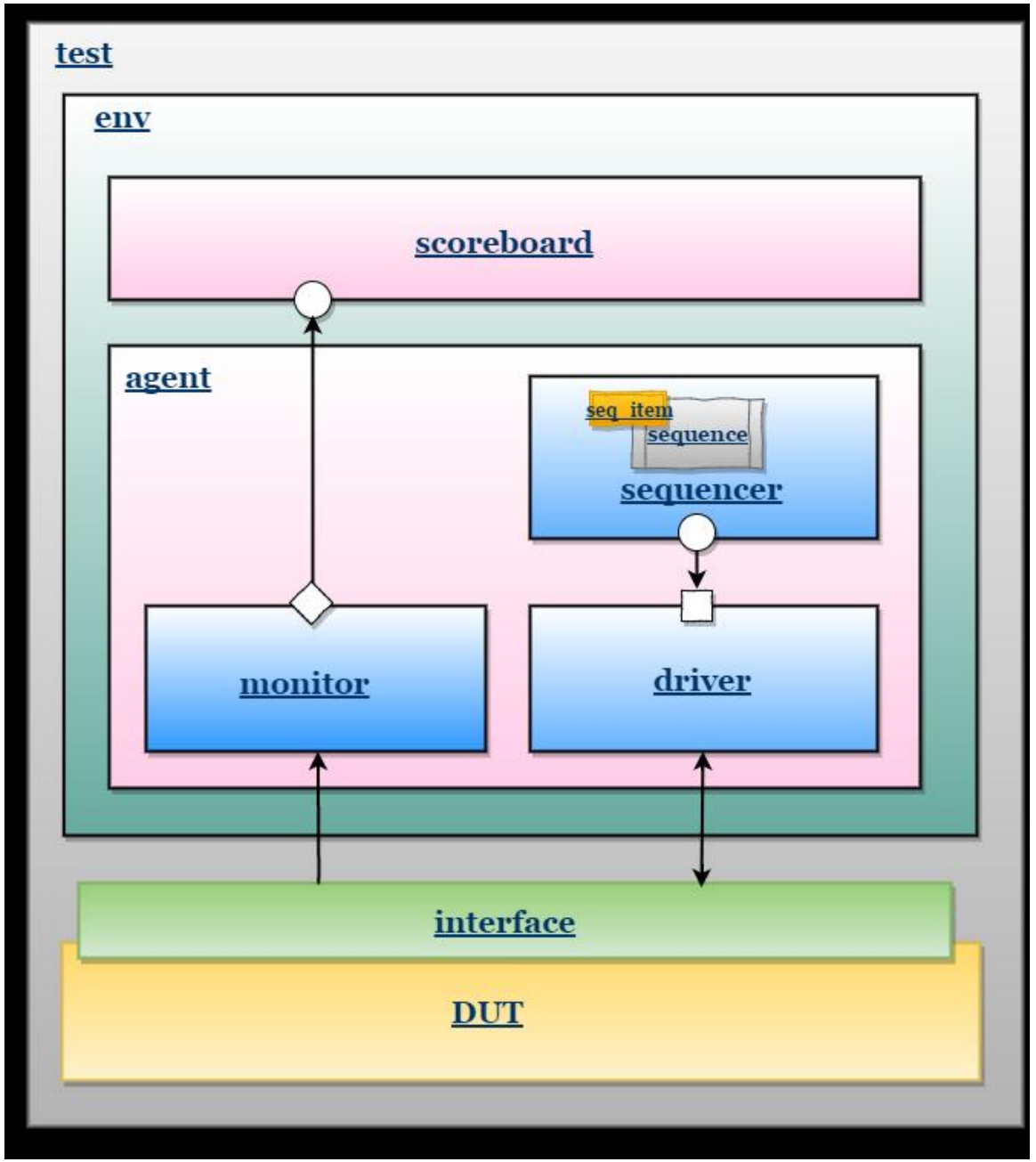
UVM Class Hierarchy



SV Testbench Architecture



UVM Test Bench Architecture



UVM Layered Test Bench Architecture

- **Layer#1: Test bench top – Connecting DUT to Test bench**
- Test bench top is container for connecting the design under test (DUT) to the test bench. Typically the HVLs provide way to encapsulate the related set of signals as interfaces. This allows passing related set of signals as single unit. Encapsulate related set of signals in to different interfaces of the design. Parameterize the interface to suit to various interface reuse needs.
- In the test bench top module connect interfaces to various design's ports. These ports are passed to the test bench. Test bench interacts with the design through these ports.

- Any other small glue logic required for connection of the design and test bench can be included in the test bench top.
- Generally test bench top ends up getting touched by multiple participants. Care should be taken to maintain the cleanliness of it.

- **Layer#2: Bus functional models – Signal interface handling**
- BFM is an engine that powers the test bench. Bus functional model (BFM) converts the signal level information to the transaction. Transaction is further processed or issued by the next layer test bench components.
- Test benches especially the object oriented programming based languages are driven by concept of “transaction and transactor”. A transaction is encapsulation of the information. Transactor executes the transaction. A transaction is like an instruction. Transactor is a like processor that processes the instruction.
- This concept is repeatedly applied in all the test bench components, even within the BFM. BFM acts as a transactor by processing transaction issued by the test bench.

- **Layer#3: Translators – Application level to BFM Transaction**
- Translators translate the application level stimulus to transaction level stimulus. Transaction level responses are converted to the application level response. The question that comes up is, what is application?
- Applications make use of designs. Real value of design is realized in application. For example a USB device design used by flash storage will utilize USB serial physical transport for transporting data based on SCSI protocol. Application level SCSI protocol runs on the USB protocol. This is made use by the operating systems to provide the file read and write services.
- End application on operating system, can generate file read and write type of requests. Drivers generate the SCSI commands corresponding for the same. The SCSI commands are translated to the USBx transaction stimulus. Here the application level stimulus of file read or write is broken down to the set of the USBx transactions. This high level stimulus to transaction level translation is grouped in as functionality of translators.
- There can be complex or even simpler translations required to convert the application level stimulus to transaction stimulus. USBx and SCSI example can be debated that SCSI to transaction translator can become part of BFM itself. BFMs themselves can raise their abstraction up to application level. When that is not the case the translators in the test bench will have to handle the conversion. A high level stimulus is generated which is more in-line with the application world. This stimulus has to be converted to the transaction interface required by the BFM using the translators.
- There can also be additional controllability added in the high level stimulus to make the process of verification easier. This controllability handling is also implemented in the translators.
- RAL and MAM aid the translators. Translators use the services of the RAL, MAM and BFM in implementing their functionality.
- Register abstraction layer (RAL) abstracts the DUT's registers. It converts the access to RAL model by test bench to the actual read & writes transactions to DUT registers. This abstraction also helps in making the address space change and physical bus change agnostic to test bench.
- Memory allocation manager (MAM) translates the memory allocation and deallocation requests to real address region allocation and deallocation. This is required for the design's requiring the system memory model.
- Scoreboard is another translator that implements the translation of the data done by DUT. The input of the DUT is captured as reference data and output of the DUT is after translation compared to the actual data.

- **Layer#4: Generators – Stimulus and Coverage**
- Generators are top-level verification components that generate both the data and control command stimulus generation. Both the data and control commands are modeled using the transactions. Transactions have constraints to constrain the variables of the transactions. These are randomized on every stimulus transaction generation.
- There are some of the parameters that get randomized only once during the initial start of the system and remains same thereafter. These are captured in structural and functional configurations. Some of the examples of the configuration parameters are minimum and maximum size of the data traffic, highest number of the ports allowed, various timeout values. These configurations are used in the constraints of the stimulus transaction generation. For example the random size of the data traffic generated is constrained between the minimum and maximum data traffic size specified in the configuration.
- Some of the stimulus generation may not only be configuration dependent but also system state dependent. For example USB host data traffic cannot start unless the device is connected and initialized. So the data traffic generator has to wait till the device is connected and initialized. Such synchronizations are achieved by maintaining the system state. State is typically maintained as shared object protected with semaphores. Its updated and used by different components of the test bench for synchronization. The information in the state object also used in the constraints of the stimulus transaction generation.
- Any constrained random testbench should also be supported with the functional coverage. Constrained random by default has the uncertainty. An intended scenario may or may not happen. So ones, which are critical or important, should be confirmed by coding the functional coverage. Functional coverage should be cleanly abstracted off separately from the test bench. This also allows easy porting of the functional coverage to other test bench environments when the need arises.

- **Layer#5: Tests – Controlling stimulus and configurations**
- The stimulus generators and configurations are the knobs provided by the test bench. These knobs are to be controlled by the tests to achieve the verification objective. Tests will control the test bench with the various knobs provided by the test bench. Tests will override the constraints of the stimulus transactions and configurations to create the scenario of interest. Test will utilize the state and events provided by the test bench to achieve the synchronization with the test bench to control and synchronize the stimulus generation.
- Tests can be either constrained random or directed in nature. Both of these forms of tests will utilize the infrastructure provided by the test bench to achieve the verification objective. Tests should be as light in terms of their code content as possible. Whenever there are additional test requirements coming in which are not satisfied by the test bench infrastructure, test bench architecture should be relooked. It indicates holes in the features provided by the test bench. Except few tests, which are testing very corner cases, all test requirements should be met by infrastructure of test bench. This is important because it prevents bloating of tests and promotes the reuse.
- Ideally tests should contain code to control test bench by APIs it provides. Test specific APIs implementation within the test should be minimized.
- Even for the checks, test should rely on the self-checks implemented in the test bench. Tests can downgrade the checks wherever they are not relevant. Only corner case scenarios where the generic self-check inside test bench is not of sufficient ROI, it should be placed in the test itself.

- UVM library contains:
  - Component classes for building testbench components like generator/driver/monitor etc.
  - Reporting classes for logging,
  - Factory for object substitution.
  - Synchronization classes for managing concurrent process.
  - Policy classes for printing, comparing, recording, packing, and unpacking of uvm_object based classes.
  - TLM Classes for transaction level interface.
  - Sequencer and Sequence classes for generating realistic stimulus.
  - And Macros which can be used for shorthand notation of complex implementation.

- **UVM simulation phases**
- UVM Components execute their behavior in strictly ordered, pre-defined phases. Each phase is defined by its own virtual method, which derived components can override to incorporate component-specific behavior. By default , these methods do nothing.

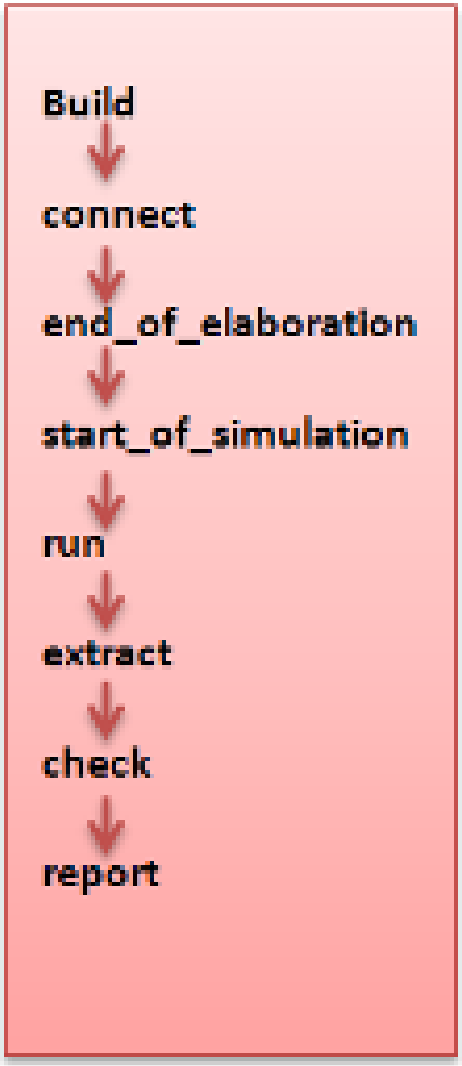
--> **virtual function void** build()
This phase is used to construct various child components/ports/exports and configures them.

--> **virtual function void** connect()
This phase is used for connecting the ports/exports of the components.

--> **virtual function void** end_of_elaboration()
This phase is used for configuring the components if require

--> **virtual function void** start_of_simulation()
This phase is used to print the banners and topology.

--> **virtual task** run()
In this phase , Main body of the test is executed where all threads are forked off.



--> **virtual function void** extract()
In this phase, all the required information is gathered.

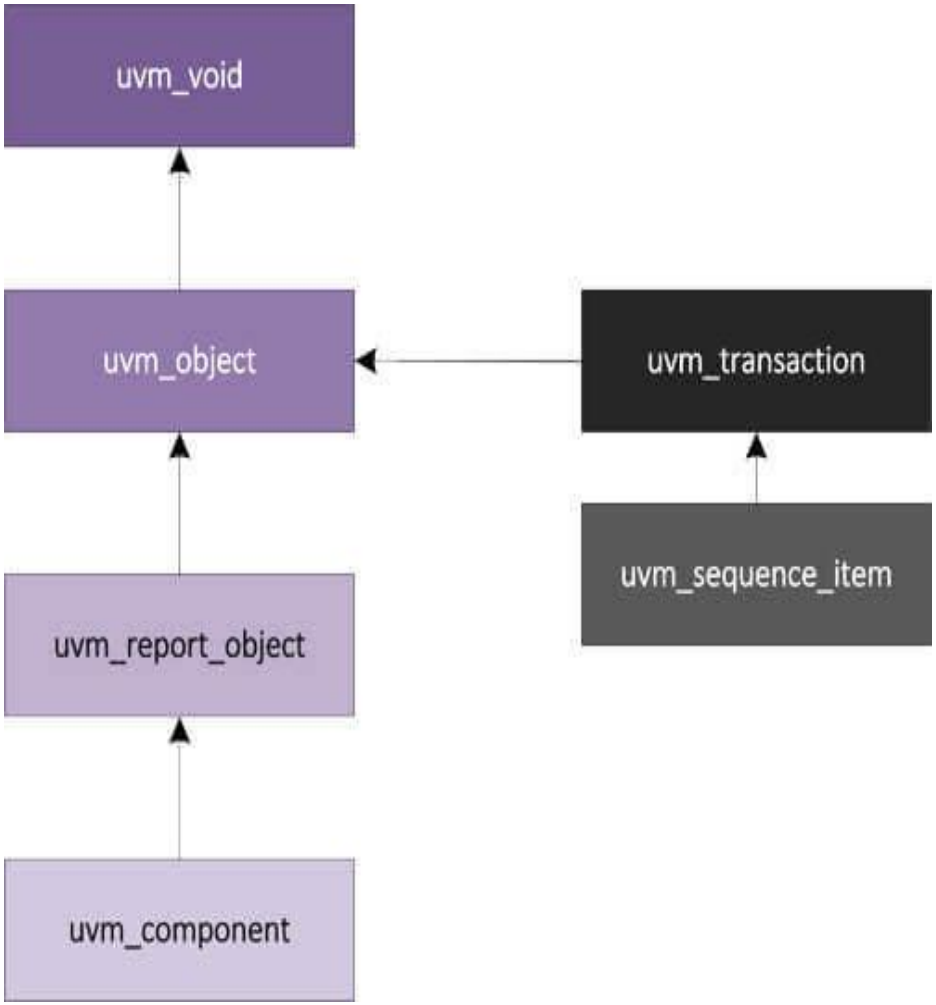
--> **virtual function void** check()
In this phase, check the results of the extracted information such as un responded requests in scoreboard, read statistics registers etc.

--> **virtual function void** report()
This phase is used for reporting the pass/fail status.

Only build() method is executed in top down manner. i.e after executing parent build() method, child objects build() methods are executed. All other methods are executed in bottom-up manner. The run() method is the only method which is time consuming. The run() method is forked, so the order in which all components run() method are executed is undefined.

Base Classes

- The basic building blocks for any verification environment are the components (drivers, sequencers, monitors ...) and the transactions (class objects that contain actual data) they use to communicate. From the UVM hierarchy, we can see that most of the classes in UVM are derived from a set of core classes that are described next



- **uvm_void**
  - This doesn't have any purpose, but serves as the base class for all UVM classes.
- **uvm_root**
  - This is an implicit top-level UVM component that is automatically created when the simulation is run, and users can access via the global (uvm_pkg-scope) variable, *uvm_top*. Please note the following properties of *uvm_top* (an instance of uvm_root)
  - Any component whose parent is set to **null** becomes a child of uvm_top.

- **uvm_object**
  - The main role of *uvm_object* is to provide a set of methods for common operations like create, copy, compare, print and record. Since all major UVCs and transactions are derived from uvm_object, all these functions are available for their use. See class definition to know more

- **uvm_report_object**
  - Provides an interface to the UVM reporting facility. All messages, warnings, errors issued by components go via this interface. There's an internal instance of uvm_report_handler which stores the reporting configuration on the basis of which the handler makes decisions on whether the message should be printed or not. Then it's passed to a central uvm_report_server which does the actual formatting and production of messages.
  - A report has ['ID String', 'Severity', 'Verbosity_Level', 'Text_Message'] and may include ['Filename', 'Line_Number'] from which the message came. If the verbosity level is greater than the configured maximum verbosity level, it is ignored. For example, if the maximum verbosity level is set to UVM_MEDIUM, and you issue a statement like `uvm_info ("STAT", "Status Registers have been updated", UVM_HIGH");, then you will not see this message in the output. By setting UVM_HIGH in the message, you are telling the report_server that you only want the message to be printed out when you want tons of messages printed out. This is useful for debug purposes, and you can set the message level to UVM_HIGH for all debug related messages, while you keep the maximum verbosity level to UVM_MEDIUM. That way you have the ability to switch between output levels.

- **uvm_component**
  - All major verification components are derived from uvm_component, and already inherits features from uvm_object and uvm_report_object. It has the following interfaces:
    - **Hierarchy** : defines methods for searching and traversing the component hierarchy. eg: env0.pci0.master1.drv
    - **Phasing** : defines a phasing system that all components can follow eg : build, connect, run, etc
    - **Reporting** : defines an interface to the Report Handler, and all messages, warnings and errors are processed through this interface (derived from uvm_report_object)
    - **Recording** : defines methods to record transactions produced/consumed by component to a transaction database.
    - **Factory** : defines an interface to the uvm_factory (used to create new components/objects based on instance/type)
  - Note that uvm_component is automatically seeded if UVM seeding is enabled, but all other objects must be manually reseeded via uvm_object::reseed.

- **uvm_transaction**
  - This is an extension of uvm_object and includes a timing and recording interface. Note that simple transactions can be derived from uvm_transaction, but sequence-enabled transactions must be derived from uvm_sequence_item.

- **UVM REPORTING/MESSAGING**

- The uvm_report_object provides an interface to the UVM reporting facility. Through this interface, components issue the various messages with different severity levels that occur during simulation. Users can configure what actions are taken and what file(s) are output for individual messages from a particular component or for all messages from all components in the environment.

  A report consists of an id string, severity, verbosity level, and the textual message itself. If the verbosity level of a report is greater than the configured maximum verbosity level of its report object, it is ignored.

  Reporting Methods:
  Following are the primary reporting methods in the UVM.

  virtual function void uvm_report_info
  (**string** id,**string** message,**int** verbosity=UVM_MEDIUM,**string** filename="",**int** line=0)

  virtual function void uvm_report_warning
  (**string** id,**string** message,**int** verbosity=UVM_MEDIUM,**string** filename="",**int** line=0)

  virtual function void uvm_report_error
  (**string** id,**string** message,**int** verbosity=UVM_LOW, **string** filename="",**int** line=0)

  virtual function void uvm_report_fatal
  (**string** id,**string** message,**int** verbosity=UVM_NONE, **string** filename="",**int** line=0)

  Arguments description:
  id -- a unique id to form a group of messages.
  message -- The message text
  verbosity -- the verbosity of the message, indicating its relative importance. If this number is less than or equal to the effective verbosity level, then the report is issued, subject to the configured action and file descriptor settings.
  filename/line -- If required to print filename and line number from where the message is issued, use macros, `__FILE__ and `__LINE__.

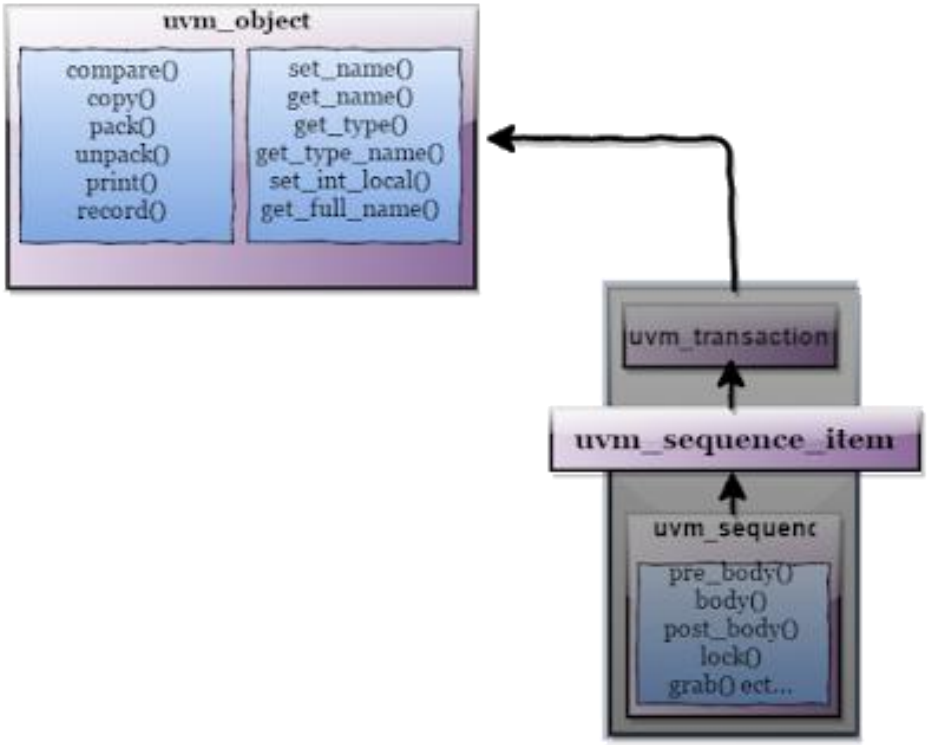
- **Actions:**
  These methods associate the specified action or actions with reports of the given severity, id, or severity-id pair.

  Following are the actions defined:

  UVM_NO_ACTION -- Do nothing
  UVM_DISPLAY -- Display report to standard output
  UVM_LOG -- Write to a file
  UVM_COUNT -- Count up to a max_quit_count value before exiting
  UVM_EXIT -- Terminates simulation immediately
  UVM_CALL_HOOK -- Callback the hook method .

UVM Sequence item

- The sequence item is written by extending the uvm_sequence_item, uvm_sequence_item inherits from the uvm_object via the uvm_transaction class.
- Therefore uvm_sequence_item is of object type.
- UVM uses the concept of a factory where all objects are registered with it so that it can return an object of the requested type when required.



- The uvm_object has a number of virtual methods which are used to implement common data object functions (copy, clone, compare, print, transaction and recording) and these should be implemented to make the sequence_item more general purpose.

- Also uvm_Object has macros defined in it, mainly Utility Macros and Field Macros.

- **Utility Macros**
  - The *utils* macro is used primarily to register an object or component with the factory and is required for it to function correctly. It is required to be used inside every user-defined class that is derived from uvm_object which includes all types of sequence items and components.
  - The utility macros provide implementations of the create method (needed for cloning) and the get_type_name method (needed for debugging), etc.
  - objects with no field macros, `uvm_object_utils(TYPE)

- **Field Macros:**
  `uvm_field_* macros that were used between *_begin and *_end utility macros are basically called field macros since they operate on class properties and provide automatic implementations of core methods like copy, compare and print.
- This helps the user save some time from implementing custom do_copy, do_compare and do_print functions for each and every class.

- The `uvm_field_* macros are invoked inside of the `uvm_*_utils_begin and `uvm_*_utils_end, for the implementations of the methods: copy, compare, pack, unpack, record, print, and etc.

  Each `uvm_field_* macro is named to correspond to a particular data type: integrals, strings, objects, queues, etc., and each has at least two arguments: FIELD and FLAG.

  `uvm_field_*(FIELD,FLAG);

- objects with field macros,
  `uvm_object_utils_begin(TYPE)
  `uvm_field_*(FIELD,FLAG)
  `uvm_object_utils_end

- However, since these macros are expanded into general code, it may impact simulation performance and are generally not recommended.

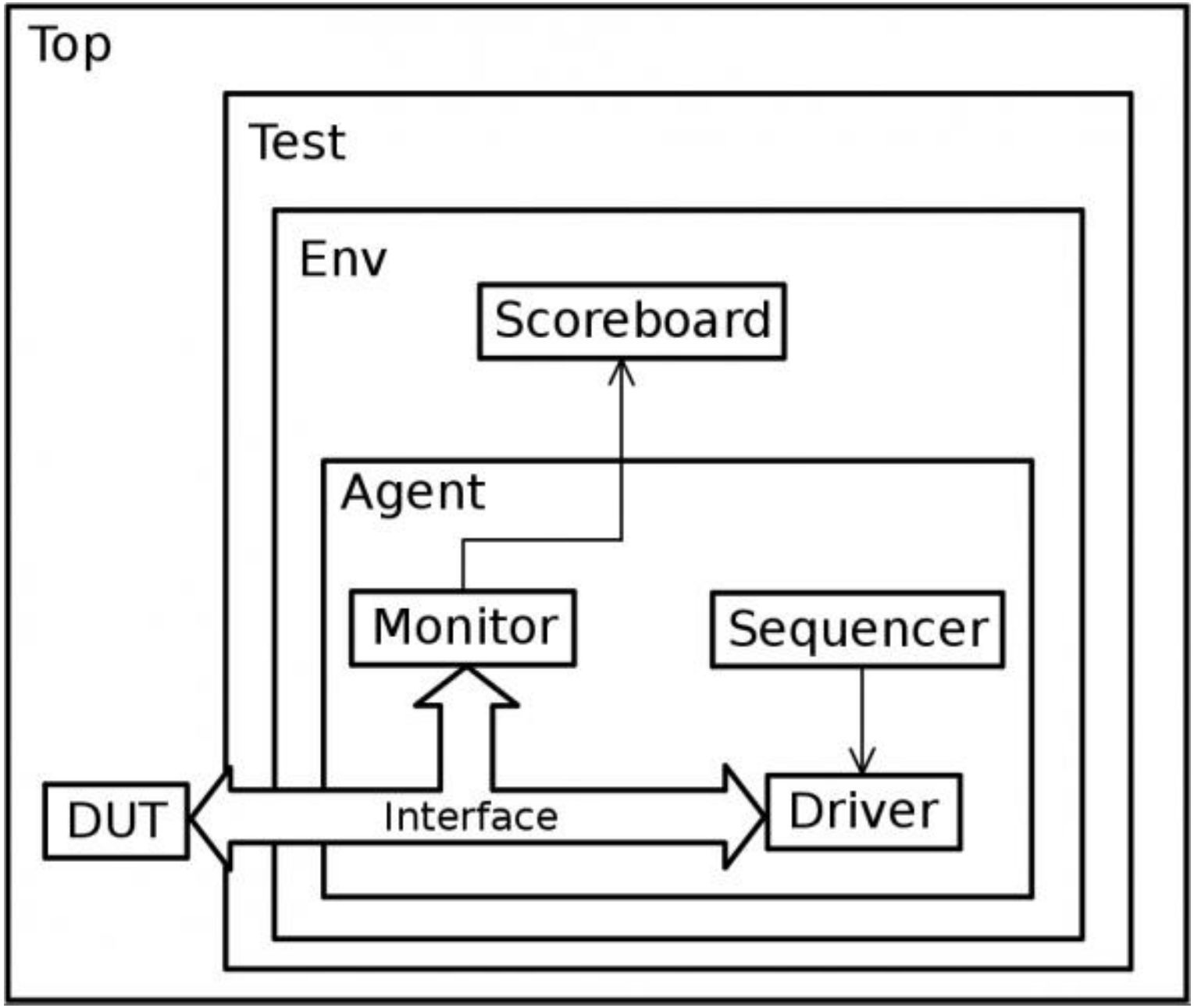
`uvm_field_*(FIELD,FLAG)

FLAG	Description
UVM_ALL_ON	Set all operations on (default).
UVM_DEFAULT	Use the default flag settings.
UVM_NOCOPY	Do not copy this field.
UVM_NOCOMPARE	Do not compare this field.
UVM_NOPRINT	Do not print this field.
UVM_NODEFPRINT	Do not print the field if it is the same as its
UVM_NOPACK	Do not pack or unpack this field.
UVM_PHYSICAL	Treat as a physical field. Use physical setting in policy class for this field.
UVM_ABSTRACT	Treat as an abstract field. Use the abstract setting in the policy class for this field.
UVM_READONLY	Do not allow setting of this field from the set_*_local methods.
A radix for printing and recording can be specified by OR'ing one of the following constants in the *FLAG* argument	
UVM_BIN	Print / record the field in binary (base-2).
UVM_DEC	Print / record the field in decimal (base-10).
UVM_UNSIGNED	Print / record the field in unsigned decimal (base-10).
UVM_OCT	Print / record the field in octal (base-8).
UVM_HEX	Print / record the field in hexadecimal (base-16).
UVM_STRING	Print / record the field in string format.
UVM_TIME	Print / record the field in time format.

UVM Environment [uvm_env]

- **What is UVM environment ?**
  - A UVM **environment** contains multiple, reusable verification components and defines their default configuration as required by the application. For example, a UVM environment may have multiple agents for different interfaces, a common scoreboard, a functional coverage collector, and additional checkers. This forms the base of any modern verification environment.

- **Why shouldn't verification components be placed directly in test class ?**
  - It is *NOT* recommended to instantiate individual verification components directly inside the test class which has the following drawbacks :
  - Test writer should know how to configure the environment
  - Changes to the topology will require updating of multiple test files, which can take a lot of time
  - Tests are not reusable because they rely on a specific environment structure
  - Hence, it is always recommended to build the testbench class from uvm_env, which can then be instantiated within multiple tests. This will allow changes in environment topology to be reflected in all the tests. Moreover, the environment should have knobs to configure, enable or disable different verification components for the desired task.


Top
Test
Env
Scoreboard
Agent
Monitor
Sequencer
DUT
Interface
Driver

- uvm_env is the base class for hierarchical containers of other components that make up a complete environment.
- It can be reused as a sub-component in a larger environment or even as a stand-alone verification environment that can instantiated directly in various tests.

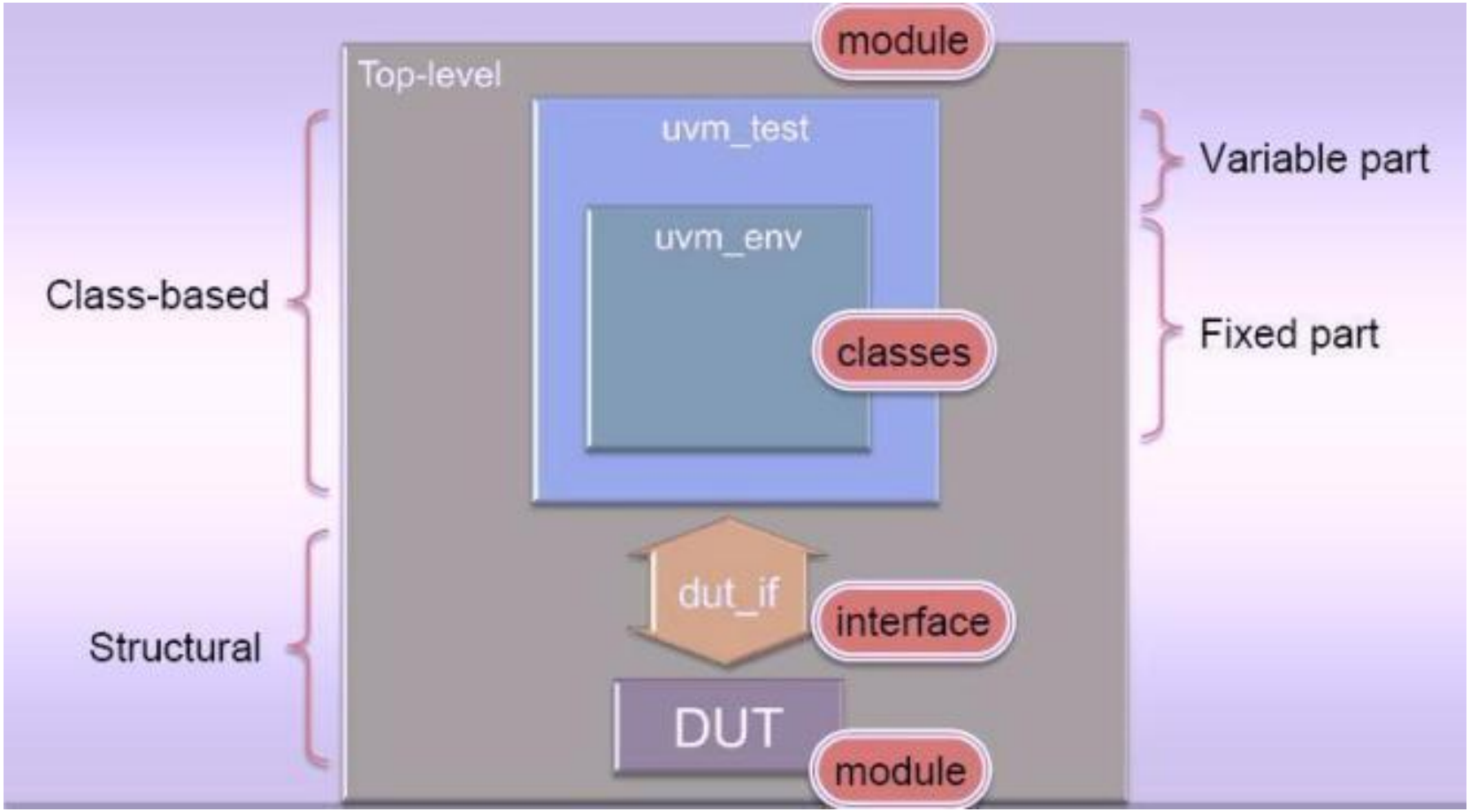
- Before understanding UVM, we need to understand verification.
- Right now, we have a DUT and we will have to interact with it in order to test its functionality, so we need to stimulate it. To achieve this, we will need a block that generates sequences of bits to be transmitted to the DUT, this block is going to be named *sequencer*.
- Usually sequencers are unaware of the communication bus, they are responsible for generating generic sequences of data and they pass that data to another block that takes care of the communication with the DUT. This block will be the *driver*.
- While the driver maintains activity with the DUT by feeding it data generated from the sequencers, it doesn't do any validation of the responses to the stimuli. We need another block that listens to the communication between the driver and the DUT and evaluates the responses from the DUT. This block is the *monitor*.
- Monitors sample the inputs and the outputs of the DUT, they try to make a prediction of the expected result and send the prediction and result of the DUT to another block, the *scoreboard*, in order to be compared and evaluated.
- All these blocks constitute a typical system used for verification and it's the same structure used for UVM testbenches.

Application of Virtual Interface and uvm_config_db

- **How to connect the DUT to the UVM Testbench..??**
- In our traditional directed Testbench environments, all the components are “static” in nature & information (data/control) is also exchanged in the form of signals/wire/net at all levels in the DUT as well as TB.

- But this is not the case in the latest “Constrained Random Verification Methodology” like UVM where DUT is static (module based) in nature yet Testbench is class (SystemVerilog OOPs) based. Hence the communication between the DUT and TB can NOT be like the one in traditional Testbenches i.e. port based connection of the DUT ports to the TB ports.

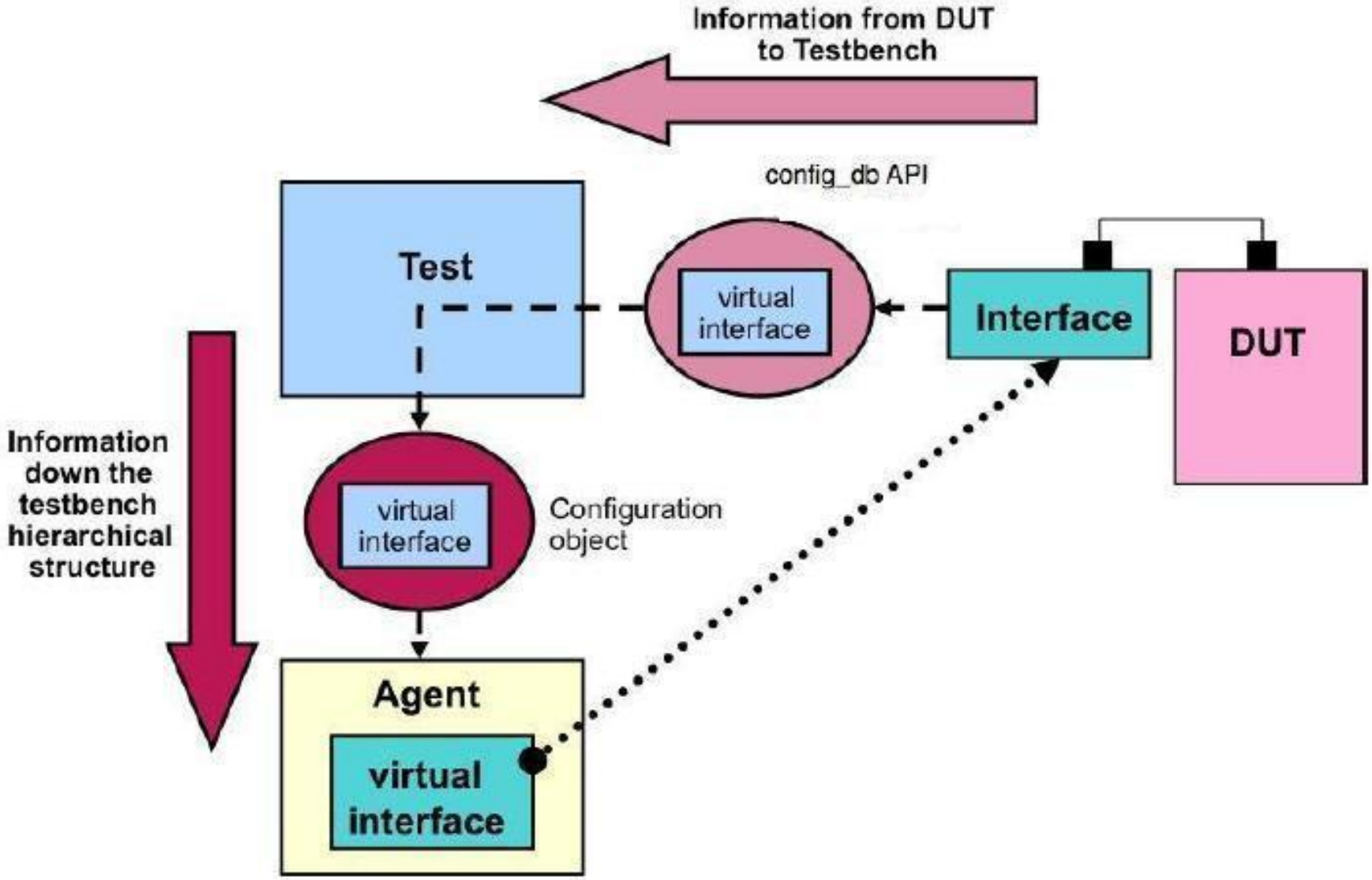
A Standard Classification of an UVM Environment



- The DUT and testbench belong to two different SystemVerilog instance worlds.
  - The DUT belongs to the static instance world while the testbench belongs to the dynamic instance world.
  - Because of this the DUT's ports can not be connected directly to the testbench class objects so a different SystemVerilog means of communication, which is virtual interfaces, is used.

- The DUT's ports are connected to an instance of an interface.
  - The Testbench communicates with the DUT through the interface instance. Using a virtual interface as a reference or handle to the interface instance, the testbench can access the tasks, functions, ports, and internal variables of the SystemVerilog interface.
  - As the interface instance is connected to the DUT pins, the testbench can monitor and control the DUT pins indirectly through the interface elements.

- The recommended approach for passing this information to the testbench is to use either the configuration database using the config_db API.


Information from DUT
to Testbench
config_db API
Test
virtual
interface
Interface
DUT
Information
down the
testbench
hierarchical
structure
virtual
interface
Configuration
object
Agent
virtual
interface

- As we can see in the diagram that the virtual interface information is provided to the Testbench (Test Class) from the DUT (*light pink color coding*) which acts as a pointer to the interface instance being used by the DUT. In actual UVM environment, the test class receives this virtual interface information from the DUT and later the test class propagates this virtual interface (interface instance handle) information to the physical component e.g. ***Driver, Monitor, Scoreboard*** etc, via a configuration object (*dark pink color coding*), which in the real sense needs this information to communicate with the DUT.